SYNAPSE Data Pipeline Design

Technical Architecture Team

March 7, 2026

Contents

1 Executive Summary **2**

2 Pipeline Architecture Overview **3**
2.1 High-Level Data Flow . . . 3
2.2 Architecture Principles . . . 3
2.2.1 Scalability . . . 3
2.2.2 Reliability . . . 3
2.2.3 Data Quality . . . 4
2.2.4 Performance . . . 4

3 Data Sources **5**
3.1 Overview . . . 5
3.2 Hacker News Firebase API . . . 5
3.2.1 API Characteristics . . . 5
3.2.2 Key Endpoints . . . 5
3.2.3 Response Schema . . . 6
3.3 GitHub REST API v3 . . . 6
3.3.1 API Characteristics . . . 6
3.3.2 Key Endpoints . . . 6
3.3.3 Response Schema . . . 6
3.4 arXiv API . . . 7
3.4.1 API Characteristics . . . 7
3.4.2 Query Endpoints . . . 7
3.4.3 Response Schema . . . 7
3.5 YouTube Data API v3 . . . 8
3.5.1 API Characteristics . . . 8
3.5.2 Key Endpoints . . . 8
3.5.3 Response Schema . . . 8
3.6 NewsAPI Integration . . . 8
3.6.1 API Characteristics . . . 8
3.6.2 Endpoints . . . 9
3.6.3 Response Schema . . . 9

4 Web Scraping Layer **10**
4.1 Scrapy Project Structure . . . 10
4.1.1 Project Organization . . . 10
4.2 Spider Designs . . . 10
4.2.1 Hacker News Spider . . . 10

4.2.2 GitHub Spider . . . 11
4.3 Playwright Integration for JavaScript-Heavy Sites . . . 12
4.4 Middleware Components . . . 13
4.4.1 Retry Middleware . . . 13
4.4.2 Proxy Rotation . . . 13
4.4.3 User-Agent Rotation . . . 13
4.5 Item Pipelines . . . 14
4.5.1 Deduplication Pipeline . . . 14
4.5.2 Validation Pipeline . . . 15

5 Message Queue System 16
5.1 Celery Architecture . . . 16
5.2 Task Definitions . . . 16
5.3 Redis Broker Configuration . . . 17
5.4 Task Routing and Priorities . . . 17
5.5 Retry Policies with Exponential Backoff . . . 18
5.6 Dead Letter Queue Management . . . 18
5.7 Flower Monitoring . . . 19

6 Data Processing Pipeline 20
6.1 HTML Cleaning and Text Normalization . . . 20
6.1.1 BeautifulSoup and html2text . . . 20
6.2 Text Normalization . . . 20
6.3 Language Detection . . . 21
6.4 Deduplication Strategy . . . 21
6.4.1 URL Hash Deduplication . . . 21
6.4.2 MinHash LSH for Near-Duplicate Detection . . . 22
6.5 Metadata Extraction . . . 22
6.6 Content Validation . . . 23

7 NLP Processing 25
7.1 spaCy Pipeline . . . 25
7.1.1 Pipeline Initialization . . . 25
7.1.2 Named Entity Recognition . . . 25
7.1.3 Dependency Parsing . . . 26
7.2 Text Summarization . . . 26
7.3 Topic Classification . . . 27
7.4 Keyword Extraction . . . 28
7.5 Sentiment Analysis . . . 29
7.6 Comprehensive NLP Pipeline . . . 29

8 Embedding Generation 31
8.1 Embedding Models . . . 31
8.1.1 OpenAI Text Embedding Ada-002 . . . 31
8.1.2 Sentence Transformers . . . 31
8.2 Batch Processing . . . 32
8.3 Rate Limiting . . . 33
8.4 pgvector Storage . . . 33

9 Storage Strategy **35**
9.1 PostgreSQL Schema 35
9.2 Redis Caching Strategy 36
9.3 S3 Media Storage 37

10 Scheduler Design **38**
10.1 Celery Beat Configuration 38
10.2 Missed Task Handling 39

11 Data Quality **40**
11.1 Validation Rules 40
11.2 Quality Scoring 41
11.3 Duplicate Detection Metrics 41

12 Open Source Libraries **43**
12.1 Core Dependencies 43
12.2 Version Management 44

13 Pipeline Monitoring **45**
13.1 Prometheus Metrics 45
13.2 Alerting Rules 46
13.3 Grafana Dashboards 46
13.4 Health Checks 48

14 Conclusion **50**

Chapter 1

Executive Summary

SYNAPSE is a comprehensive data intelligence platform designed to aggregate, process, and analyze information from multiple web sources including Hacker News, arXiv, GitHub, and YouTube. This document provides detailed technical specifications for the data pipeline architecture, covering ingestion, processing, storage, and monitoring components.

The pipeline processes tens of thousands of items daily through a distributed task queue system, applies advanced NLP transformations, and stores results in a multi-tier storage infrastructure optimized for search and retrieval performance.

Chapter 2

Pipeline Architecture Overview

2.1 High-Level Data Flow

The SYNAPSE data pipeline follows a modular, event-driven architecture with distinct layers:

1. **Data Ingestion Layer**: Web scrapers and API clients collect raw data from external sources
2. **Message Queue**: Celery/Redis broker manages asynchronous task distribution
3. **Data Processing Layer**: Deduplication, cleaning, validation, and metadata extraction
4. **NLP Processing Layer**: Entity recognition, summarization, classification, sentiment analysis
5. **Embedding Generation**: Converting text to vector representations
6. **Storage Layer**: PostgreSQL, pgvector, Redis caching, S3 media storage
7. **Indexing Layer**: Full-text search indexes, vector similarity search

2.2 Architecture Principles

The pipeline is designed with the following principles:

2.2.1 Scalability

Celery workers can be horizontally scaled to handle increasing data volume. The Redis broker distributes tasks across multiple worker processes. PostgreSQL with pgvector handles large-scale vector storage.

2.2.2 Reliability

Task retry mechanisms with exponential backoff ensure transient failures don't cause data loss. Dead letter queues capture persistent failures for manual review. Monitoring and alerting detect anomalies early.

2.2.3 Data Quality

Multiple validation stages check data integrity. Deduplication prevents redundant storage. Quality scoring tracks data reliability. Comprehensive logging enables auditing.

2.2.4 Performance

Caching strategies reduce database load for hot data. Batch processing optimizes resource utilization. Asynchronous task execution prevents blocking operations.

Chapter 3

Data Sources

3.1 Overview

SYNAPSE integrates data from five primary sources, each with distinct API characteristics, authentication requirements, and rate limits. Data collection is coordinated through a scheduling system that respects source limitations while maximizing coverage.

3.2 Hacker News Firebase API

3.2.1 API Characteristics

Hacker News provides real-time access to stories, comments, and user data through an unofficial Firebase API endpoint: `https://hacker-news.firebaseio.com/v0/`

- **Authentication**: None required (public API)
- **Rate Limit**: Effectively unlimited (no official limit)
- **Response Format**: JSON
- **Update Frequency**: Real-time

3.2.2 Key Endpoints

- `/topstories.json`: Returns top 500 story IDs (sorted by ranking)
- `/beststories.json`: Returns best 500 story IDs
- `/newestories.json`: Returns newest 500 story IDs
- `/item/[id].json`: Returns item details (stories, comments, jobs)
- `/user/[id].json`: Returns user profile information

3.2.3 Response Schema

Each story object contains:

```
{
  "id": 12345678,
  "type": "story",
  "by": "username",
  "time": 1234567890,
  "title": "Story␣Title",
  "url": "https://example.com",
  "text": "Optional␣text␣content",
  "score": 250,
  "descendants": 45,
  "kids": [12345679, 12345680]
}
```

3.3 GitHub REST API v3

3.3.1 API Characteristics

GitHub provides comprehensive repository, issue, and user data through a well-documented REST API.

- **Authentication**: OAuth2 token-based (required for higher rates)
- **Rate Limit**: 5000 requests/hour (authenticated), 60 requests/hour (unauthenticated)
- **Response Format**: JSON
- **Pagination**: Cursor-based with per_page parameter

3.3.2 Key Endpoints

- `/search/repositories`: Search repositories by criteria
- `/repos/[owner]/[repo]`: Get repository details
- `/repos/[owner]/[repo]/issues`: List issues and pull requests
- `/repos/[owner]/[repo]/releases`: List releases
- `/users/[username]`: Get user profile

3.3.3 Response Schema

Repository object structure:

```
{
  "id": 1234567,
  "name": "repo-name",
  "full_name": "owner/repo-name",
  "owner": {"login": "username", "type": "User"},
  "description": "Repository␣description",
  "url": "https://api.github.com/repos/owner/repo",
  "html_url": "https://github.com/owner/repo",
  "stars_count": 1500,
  "forks_count": 250,
  "language": "Python",
  "updated_at": "2024-01-15T10:30:00Z"
}
```

3.4 arXiv API

3.4.1 API Characteristics

arXiv provides access to preprints in physics, mathematics, computer science, and related fields.

- **Authentication**: None required
- **Rate Limit**: No more than 3 requests per second
- **Response Format**: Atom XML
- **Update Frequency**: Daily

3.4.2 Query Endpoints

- `http://export.arxiv.org/api/query`: Main query interface
- Query parameters: `search_query`, `start`, `max_results`, `sortBy`, `sortOrder`

3.4.3 Response Schema

Entry element in Atom feed:

```
<entry>
  <id>http://arxiv.org/abs/2401.12345v1</id>
  <title>Paper Title</title>
  <summary>Abstract text...</summary>
  <author><name>Author Name</name></author>
  <published>2024-01-15T00:00:00Z</published>
  <updated>2024-01-15T00:00:00Z</updated>
  <link href="http://arxiv.org/pdf/2401.12345v1" title="pdf"/>
  <arxiv:primary_category term="cs.AI"/>
</entry>
```

3.5 YouTube Data API v3

3.5.1 API Characteristics

YouTube provides access to video metadata, channels, and search results through the Data API.

- **Authentication**: API key or OAuth2 token required
- **Rate Limit**: 10,000 quota units per day
- **Response Format**: JSON
- **Search Results**: Limited to 1000 results per query

3.5.2 Key Endpoints

- `/search`: Search videos, channels, playlists
- `/videos`: Get video metadata and statistics
- `/channels`: Get channel information
- `/captions`: Get video captions (requires authentication)

3.5.3 Response Schema

Video search result:

```
{
  "kind": "youtube#searchResult",
  "etag": "abc123",
  "id": {
    "kind": "youtube#video",
    "videoId": "dQw4w9WgXcQ"
  },
  "snippet": {
    "publishedAt": "2024-01-15T10:30:00Z",
    "title": "Video␣Title",
    "description": "Video␣description",
    "thumbnails": {...},
    "channelTitle": "Channel␣Name"
  }
}
```

3.6 NewsAPI Integration

3.6.1 API Characteristics

NewsAPI aggregates news articles from multiple sources worldwide.

- **Authentication**: API key required

- **Rate Limit**: 100 requests per day (free tier)
- **Response Format**: JSON
- **Coverage**: 38,000+ sources

3.6.2 Endpoints

- `/everything`: Search all articles across all time
- `/top-headlines`: Get top headlines by category/country

3.6.3 Response Schema

Article object:

```
{
  "source": {"id": "techcrunch", "name": "TechCrunch"},
  "author": "Author␣Name",
  "title": "Article␣Title",
  "description": "Article␣description",
  "url": "https://techcrunch.com/article",
  "urlToImage": "https://...",
  "publishedAt": "2024-01-15T10:30:00Z",
  "content": "Article␣full␣content..."
}
```

Chapter 4

Web Scraping Layer

4.1 Scrapy Project Structure

The web scraping infrastructure uses Scrapy 2.11 as the primary framework for structured data extraction.

4.1.1 Project Organization

```
synapse_scraper/
  scrapy.cfg
  synapse_scraper/
    __init__.py
    settings.py
    pipelines.py
    middlewares.py
    spiders/
      __init__.py
      hackernews_spider.py
      github_spider.py
      arxiv_spider.py
      youtube_spider.py
      newsapi_spider.py
    items.py
    loaders.py
```

4.2 Spider Designs

4.2.1 Hacker News Spider

The HN spider collects stories and comments from the Firebase API endpoint. It implements paginated collection with comment threading.

```
class HackerNewsSpider(scrapy.Spider):
    name = 'hackernews'
    allowed_domains = ['hacker-news.firebaseio.com']
    start_urls = [
        'https://hacker-news.firebaseio.com/v0/topstories.json'
```

```
    ]

    custom_settings = {
        'CONCURRENT_REQUESTS': 16,
        'DOWNLOAD_DELAY': 1
    }

    def parse(self, response):
        story_ids = json.loads(response.text)[:30]
        for story_id in story_ids:
            url = (f'https://hacker-news.firebaseio.com/'
                   f'v0/item/{story_id}.json')
            yield scrapy.Request(
                url,
                callback=self.parse_story
            )

    def parse_story(self, response):
        data = json.loads(response.text)
        item = HNItem()
        item['source'] = 'hackernews'
        item['external_id'] = str(data.get('id'))
        item['title'] = data.get('title', '')
        item['url'] = data.get('url', '')
        item['author'] = data.get('by', '')
        item['score'] = data.get('score', 0)
        item['comments_count'] = data.get('descendants', 0)
        item['published_at'] = datetime.fromtimestamp(
            data.get('time', 0)
        )
        item['extracted_at'] = datetime.utcnow()
        yield item
```

4.2.2 GitHub Spider

The GitHub spider uses REST API pagination to collect trending repositories and recent releases.

```
class GitHubSpider(scrapy.Spider):
    name = 'github'
    allowed_domains = ['api.github.com']

    custom_settings = {
        'CONCURRENT_REQUESTS': 8,
        'DOWNLOAD_DELAY': 0.5
    }

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.token = settings.get('GITHUB_API_TOKEN')
```

```
    def start_requests(self):
        headers = {'Authorization': f'token␣{self.token}'}
        url = ('https://api.github.com/search/repositories'
               '?q=stars:>1000'
               '&sort=stars'
               '&order=desc'
               '&per_page=100')
        yield scrapy.Request(url, headers=headers,
                             callback=self.parse)

    def parse(self, response):
        data = json.loads(response.text)
        for repo in data.get('items', []):
            item = GitHubItem()
            item['source'] = 'github'
            item['external_id'] = str(repo['id'])
            item['name'] = repo['name']
            item['url'] = repo['html_url']
            item['owner'] = repo['owner']['login']
            item['stars'] = repo['stargazers_count']
            item['language'] = repo.get('language', '')
            item['description'] = repo.get('description', '')
            item['extracted_at'] = datetime.utcnow()
            yield item
```

4.3 Playwright Integration for JavaScript-Heavy Sites

For sites requiring JavaScript execution, Playwright is integrated through Scrapy middleware.

```
class PlaywrightDownloadHandler:
    def __init__(self):
        self.browser = None

    async def download_request(self, request, spider):
        if not self.browser:
            self.browser = await async_playwright().start()

        page = await self.browser.new_page()
        try:
            await page.goto(request.url, wait_until='load')
            await page.wait_for_selector(
                '[data-testid="tweet"]',
                timeout=10000
            )
            content = await page.content()
            return HtmlResponse(
                url=request.url,
                body=content.encode('utf-8'),
                encoding='utf-8',
                request=request
```

```
            )
        finally:
            await page.close()
```

4.4 Middleware Components

4.4.1 Retry Middleware

Custom retry logic with exponential backoff handles transient failures.

```
class RetryMiddleware:
    def __init__(self):
        self.retry_times = {}

    def process_response(self, request, response, spider):
        if response.status in [429, 500, 502, 503, 504]:
            if request.url not in self.retry_times:
                self.retry_times[request.url] = 0

            retry_count = self.retry_times[request.url]
            if retry_count < 5:
                delay = (2 ** retry_count) + random.uniform(0, 1)
                self.retry_times[request.url] += 1
                return request.replace(
                    dont_obey_robotstxt=True,
                    dont_retry=False
                )
        return response
```

4.4.2 Proxy Rotation

Rotates proxy servers to distribute load and avoid IP blocking.

```
class ProxyRotationMiddleware:
    def __init__(self, proxy_list):
        self.proxy_list = proxy_list
        self.current_index = 0

    def process_request(self, request, spider):
        proxy = self.proxy_list[
            self.current_index % len(self.proxy_list)
        ]
        request.meta['proxy'] = proxy
        self.current_index += 1
```

4.4.3 User-Agent Rotation

Rotates user-agent strings to appear as different browsers.

```
class UserAgentMiddleware:
    USER_AGENTS = [
```

```
        'Mozilla/5.0 (Windows NT 10.0; Win64; x64) '
        'AppleWebKit/537.36 (KHTML, like Gecko) '
        'Chrome/120.0.0.0 Safari/537.36',
        'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) '
        'AppleWebKit/537.36 (KHTML, like Gecko) '
        'Chrome/120.0.0.0 Safari/537.36',
        'Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 '
        '(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36'
    ]

    def process_request(self, request, spider):
        request.headers['User-Agent'] = random.choice(
            self.USER_AGENTS
        )
```

4.5 Item Pipelines

4.5.1 Deduplication Pipeline

Deduplication prevents storing duplicate items using multiple strategies.

```
from hashlib import sha256
from datasketch import MinHash

class DeduplicationPipeline:
    def __init__(self):
        self.seen_urls = set()
        self.url_hashes = set()
        self.minhash_cache = {}

    def process_item(self, item, spider):
        url_hash = sha256(
            item.get('url', '').encode()
        ).hexdigest()

        if url_hash in self.url_hashes:
            raise DropItem(
                f"Duplicate item found: {item['url']}"
            )

        self.url_hashes.add(url_hash)

        if 'content' in item:
            minhash = MinHash(num_perm=128)
            tokens = item['content'].split()
            for token in tokens:
                minhash.update(token.encode('utf8'))

            item['content_hash'] = minhash

        return item
```

4.5.2 Validation Pipeline

Validates data quality and completeness before storage.

```
class ValidationPipeline:
    REQUIRED_FIELDS = {
        'source', 'external_id', 'title', 'extracted_at'
    }

    def process_item(self, item, spider):
        missing_fields = (
            self.REQUIRED_FIELDS - set(item.keys())
        )
        if missing_fields:
            raise DropItem(
                f"Missing␣fields:␣{missing_fields}"
            )

        if len(item.get('title', '')) < 5:
            raise DropItem("Title␣too␣short")

        return item
```

Chapter 5

Message Queue System

5.1 Celery Architecture

Celery 5.3 provides distributed task queue functionality with Redis as the message broker. The system handles asynchronous processing of scraped items, NLP transformations, and storage operations.

5.2 Task Definitions

Core task definitions for the data pipeline:

```
from celery import Celery, group, chain
from celery.schedules import crontab

app = Celery('synapse')
app.config_from_object('celeryconfig')

@app.task(bind=True, max_retries=5)
def process_scraped_item(self, item_data):
    try:
        item = Item.objects.create(**item_data)
        return {'status': 'processed', 'item_id': item.id}
    except Exception as exc:
        retry_delay = (2 ** self.request.retries) + 1
        raise self.retry(exc=exc, countdown=retry_delay)

@app.task(bind=True, max_retries=3)
def nlp_process_item(self, item_id):
    item = Item.objects.get(id=item_id)
    try:
        result = process_nlp(item.content)
        item.entities = result['entities']
        item.summary = result['summary']
        item.save()
        return {'item_id': item_id, 'status': 'nlp_complete'}
    except Exception as exc:
        raise self.retry(exc=exc, countdown=60)
```

```python
@app.task(max_retries=3)
def generate_embeddings(item_id, embedding_model='ada'):
    item = Item.objects.get(id=item_id)
    vector = generate_embedding(item.title, item.content)
    ItemEmbedding.objects.create(
        item=item,
        vector=vector,
        model=embedding_model
    )
    return {'item_id': item_id, 'vector_dim': len(vector)}

@app.task
def schedule_scraper_task(spider_name):
    from scrapy.crawler import CrawlerProcess
    process = CrawlerProcess()
    process.crawl(spider_name)
    process.start()
```

5.3 Redis Broker Configuration

```python
CELERY_BROKER_URL = 'redis://localhost:6379/0'
CELERY_RESULT_BACKEND = 'redis://localhost:6379/1'
CELERY_ACCEPT_CONTENT = ['json']
CELERY_TASK_SERIALIZER = 'json'
CELERY_RESULT_SERIALIZER = 'json'
CELERY_TIMEZONE = 'UTC'
CELERY_ENABLE_UTC = True

CELERY_TASK_ROUTES = {
    'tasks.process_scraped_item': {'queue': 'scraping'},
    'tasks.nlp_process_item': {'queue': 'nlp'},
    'tasks.generate_embeddings': {'queue': 'embeddings'},
}

CELERY_DEFAULT_QUEUE = 'default'
CELERY_QUEUES = (
    Queue('default', Exchange('default'), routing_key='default'),
    Queue('scraping', Exchange('scraping'),
       routing_key='scraping'),
    Queue('nlp', Exchange('nlp'), routing_key='nlp'),
    Queue('embeddings', Exchange('embeddings'),
       routing_key='embeddings'),
    Queue('dlq', Exchange('dlq'), routing_key='dlq'),
)
```

5.4 Task Routing and Priorities

Tasks are routed to specialized queues based on function and priority:

```
CELERY_TASK_DEFAULT_PRIORITY = 5
CELERY_TASK_DEFAULT_RATE_LIMIT = '1000/m'

CELERY_TASK_QUEUE_PRIORITIES = {
    'scraping': 5,
    'nlp': 7,
    'embeddings': 3,
}

app.conf.task_routes = {
    'tasks.urgent_notification': {'queue': 'high_priority'},
    'tasks.generate_embeddings': {'queue': 'embeddings'},
    'tasks.nlp_process_item': {'queue': 'nlp'},
}
```

5.5 Retry Policies with Exponential Backoff

```
from celery.utils.log import get_task_logger
from datetime import timedelta

logger = get_task_logger(__name__)

@app.task(
    bind=True,
    max_retries=10,
    default_retry_delay=60,
    autoretry_for=(Exception,),
    retry_kwargs={'max_retries': 10}
)
def resilient_task(self, data):
    try:
        process_data(data)
        logger.info(f"Task completed for data: {data}")
    except Exception as exc:
        logger.error(f"Task failed: {exc}, Retry
           {self.request.retries}")
        retry_count = self.request.retries
        countdown = (2 ** min(retry_count, 8)) + 1
        raise self.retry(exc=exc, countdown=countdown)
```

5.6 Dead Letter Queue Management

Failed tasks after maximum retries are sent to DLQ for manual inspection:

```
@app.task(bind=True)
def send_to_dlq(self, task_name, args, exc):
    dlq_item = DeadLetterItem.objects.create(
        task_name=task_name,
        args=json.dumps(args),
```

```
        exception=str(exc),
        timestamp=datetime.utcnow(),
        status='pending_review'
    )
    logger.error(
        f"Task␣{task_name}␣sent␣to␣DLQ:␣{dlq_item.id}"
    )
    return {'dlq_id': dlq_item.id}

@app.task
def process_dlq_items():
    dlq_items = DeadLetterItem.objects.filter(
        status='pending_review'
    ).order_by('timestamp')[:100]

    for item in dlq_items:
        try:
            task_class = get_task_by_name(item.task_name)
            args = json.loads(item.args)
            task_class.apply_async(args=args)
            item.status = 'retried'
            item.save()
        except Exception as e:
            logger.error(f"DLQ␣retry␣failed:␣{e}")
            item.status = 'failed'
            item.save()
```

5.7 Flower Monitoring

Flower provides real-time monitoring of Celery workers and tasks:

```
# Run Flower for monitoring
# celery -A synapse flower --port=5555

# Flower configuration in settings
FLOWER_PORT = 5555
FLOWER_BASIC_AUTH = ['admin:password']
FLOWER_MAX_TASKS = 10000
FLOWER_PERSISTENT = True
FLOWER_DB = 'flower.db'
FLOWER_TASKS_MAX_LENGTH = 5000
```

Chapter 6

Data Processing Pipeline

6.1 HTML Cleaning and Text Normalization

Raw HTML content from web sources is cleaned and normalized to extract structured text.

6.1.1 BeautifulSoup and html2text

```
from bs4 import BeautifulSoup
import html2text

def clean_html_content(html_content):
    soup = BeautifulSoup(html_content, 'html.parser')

    for script in soup(['script', 'style']):
        script.decompose()

    for element in soup.find_all(['nav', 'footer']):
        element.decompose()

    h = html2text.HTML2Text()
    h.ignore_links = False
    h.ignore_images = True
    h.ignore_emphasis = False

    text = h.handle(str(soup))
    text = '␣'.join(text.split())

    return text
```

6.2 Text Normalization

Text normalization handles encoding, whitespace, and special characters:

```
import re
import unicodedata
```

```
def normalize_text(text):
    text = unicodedata.normalize('NFKD', text)
    text = text.encode('ascii', 'ignore').decode('ascii')

    text = re.sub(r'\s+', '␣', text)
    text = re.sub(r'[^\w\s\.\,\!\?\-]', '', text)
    text = text.strip()

    return text

def clean_and_normalize(raw_content):
    cleaned = clean_html_content(raw_content)
    normalized = normalize_text(cleaned)
    return normalized
```

6.3 Language Detection

Using langdetect library to identify document language:

```
from langdetect import detect, detect_langs
from langdetect.lang_detect_exception import LangDetectException

def detect_language(text):
    try:
        if len(text) < 10:
            return None

        langs = detect_langs(text)
        primary_lang = langs[0]

        if primary_lang.prob > 0.95:
            return primary_lang.lang
        else:
            return None
    except LangDetectException:
        return None

def filter_by_language(text, target_langs=['en']):
    lang = detect_language(text)
    return lang in target_langs if lang else False
```

6.4 Deduplication Strategy

Multi-level deduplication prevents storing duplicate content:

6.4.1 URL Hash Deduplication

```
from hashlib import sha256
```

```
def get_url_hash(url):
    url_lower = url.lower().strip()
    return sha256(url_lower.encode()).hexdigest()

def check_url_duplicate(url, seen_hashes):
    url_hash = get_url_hash(url)
    return url_hash in seen_hashes
```

6.4.2 MinHash LSH for Near-Duplicate Detection

```
from datasketch import MinHash, MinHashLSH

class NearDuplicateDetector:
    def __init__(self, num_perm=128, threshold=0.5):
        self.num_perm = num_perm
        self.threshold = threshold
        self.lsh = MinHashLSH(
            threshold=threshold,
            num_perm=num_perm
        )

    def text_to_minhash(self, text):
        m = MinHash(num_perm=self.num_perm)
        for token in text.split():
            m.update(token.encode('utf8'))
        return m

    def is_near_duplicate(self, text, text_id):
        mh = self.text_to_minhash(text)
        results = self.lsh.query(mh)

        if results:
            return True, results

        self.lsh.insert(text_id, mh)
        return False, []
```

6.5 Metadata Extraction

```
from dateutil import parser as date_parser
import re

class MetadataExtractor:
    @staticmethod
    def extract_date(text, fallback_date=None):
        date_patterns = [
            r'\d{4}-\d{2}-\d{2}',
            r'\d{2}/\d{2}/\d{4}',
            r'[A-Za-z]+␣\d{1,2},␣\d{4}'
```

```
        ]

        for pattern in date_patterns:
            matches = re.findall(pattern, text)
            if matches:
                try:
                    return date_parser.parse(matches[0])
                except:
                    continue

        return fallback_date

    @staticmethod
    def extract_author(item):
        author_candidates = [
            item.get('author'),
            item.get('by'),
            item.get('creator'),
            item.get('username')
        ]
        return next(
            (a for a in author_candidates if a),
            'Unknown'
        )

    @staticmethod
    def extract_source(url):
        from urllib.parse import urlparse
        parsed = urlparse(url)
        return parsed.netloc.replace('www.', '')
```

6.6 Content Validation

```
class ContentValidator:
    MIN_TITLE_LENGTH = 5
    MAX_TITLE_LENGTH = 500
    MIN_CONTENT_LENGTH = 50
    MAX_CONTENT_LENGTH = 100000

    @classmethod
    def validate_title(cls, title):
        if not title:
            return False, "Title is empty"

        if len(title) < cls.MIN_TITLE_LENGTH:
            return False, "Title too short"

        if len(title) > cls.MAX_TITLE_LENGTH:
            return False, "Title too long"
```

```
        return True, "Valid"

    @classmethod
    def validate_content(cls, content):
        if not content:
            return False, "Content is empty"

        if len(content) < cls.MIN_CONTENT_LENGTH:
            return False, "Content too short"

        if len(content) > cls.MAX_CONTENT_LENGTH:
            return False, "Content too long"

        return True, "Valid"

    @classmethod
    def validate_item(cls, item):
        validations = [
            cls.validate_title(item.get('title')),
            cls.validate_content(item.get('content')),
        ]

        for is_valid, message in validations:
            if not is_valid:
                return False, message

        return True, "All validations passed"
```

Chapter 7

NLP Processing

7.1 spaCy Pipeline

spaCy 3.7 provides efficient natural language processing with pre-trained models.

7.1.1 Pipeline Initialization

```
import spacy
from spacy.language import Language

@Language.factory("synapse_nlp")
def create_nlp_pipeline(nlp, name):
    pipe = nlp.add_pipe("synapse_processor", last=True)
    return pipe

nlp = spacy.load("en_core_web_sm")
nlp.add_pipe("synapse_processor", last=True)

def process_text(text):
    doc = nlp(text)
    return doc
```

7.1.2 Named Entity Recognition

Entity recognition identifies people, organizations, locations, and technical terms:

```
def extract_entities(doc):
    entities = []
    for ent in doc.ents:
        entities.append({
            'text': ent.text,
            'label': ent.label_,
            'start': ent.start_char,
            'end': ent.end_char
        })
    return entities

def extract_tech_entities(text):
```

```
    tech_terms = [
        'Python', 'JavaScript', 'SQL', 'Docker', 'Kubernetes',
        'React', 'Vue', 'Angular', 'Machine␣Learning', 'AI'
    ]

    found_terms = []
    for term in tech_terms:
        if term.lower() in text.lower():
            found_terms.append(term)

    return found_terms
```

7.1.3 Dependency Parsing

```
def extract_dependencies(doc):
    deps = []
    for token in doc:
        deps.append({
            'text': token.text,
            'dep': token.dep_,
            'head': token.head.text,
            'pos': token.pos_
        })
    return deps

def extract_subjects_objects(doc):
    subjects = []
    objects = []

    for token in doc:
        if token.dep_ in ['nsubj', 'nsubjpass']:
            subjects.append(token.text)
        elif token.dep_ in ['dobj', 'pobj']:
            objects.append(token.text)

    return subjects, objects
```

7.2 Text Summarization

Using HuggingFace transformers for abstractive summarization:

```
from transformers import pipeline

summarizer = pipeline(
    "summarization",
    model="facebook/bart-large-cnn",
    device=0
)

def generate_summary(text, max_length=150, min_length=50):
```

```
    if len(text.split()) < 100:
        return text

    summary = summarizer(
        text,
        max_length=max_length,
        min_length=min_length,
        do_sample=False
    )

    return summary[0]['summary_text']

def batch_summarize(texts, batch_size=8):
    summaries = []
    for i in range(0, len(texts), batch_size):
        batch = texts[i:i+batch_size]
        for text in batch:
            summary = generate_summary(text)
            summaries.append(summary)
    return summaries
```

7.3 Topic Classification

Zero-shot classification for flexible topic assignment:

```
from transformers import pipeline

zero_shot_classifier = pipeline(
    "zero-shot-classification",
    model="facebook/bart-large-mnli"
)

TOPIC_LABELS = [
    'Machine␣Learning', 'Web␣Development', 'DevOps',
    'Data␣Science', 'Security', 'Cloud␣Computing',
    'Mobile␣Development', 'Game␣Development'
]

def classify_topics(text, labels=TOPIC_LABELS):
    result = zero_shot_classifier(
        text,
        labels,
        multi_class=True
    )

    return [
        {'label': label, 'score': score}
        for label, score in zip(result['labels'],
           result['scores'])
    ]
```

7.4 Keyword Extraction

Using KeyBERT and YAKE for keyword extraction:

```
from keybert import KeyBERT
import yake

kb_model = KeyBERT(model='all-MiniLM-L6-v2')

def extract_keywords_keybert(text, language='english', top_n=10):
    keywords = kb_model.extract_keywords(
        text,
        language=language,
        top_n=top_n,
        use_maxsum=True,
        nr_candidates=20
    )

    return [
        {'keyword': kw, 'score': score}
        for kw, score in keywords
    ]

def extract_keywords_yake(text, top_n=10):
    kw_extractor = yake.KeywordExtractor(
        lan='en',
        n=3,
        top=top_n,
        features=None
    )

    keywords = kw_extractor.extract_keywords(text)
    return [
        {'keyword': kw, 'score': score}
        for kw, score in keywords
    ]

def extract_combined_keywords(text, top_n=10):
    keybert_results = extract_keywords_keybert(text, top_n=top_n)
    yake_results = extract_keywords_yake(text, top_n=top_n)

    combined = {}
    for item in keybert_results + yake_results:
        kw = item['keyword']
        if kw not in combined:
            combined[kw] = 0
        combined[kw] += item['score']

    sorted_keywords = sorted(
        combined.items(),
        key=lambda x: x[1],
        reverse=True
```

```
    )[:top_n]

    return [{'keyword': kw, 'score': score}
            for kw, score in sorted_keywords]
```

7.5 Sentiment Analysis

Using pre-trained sentiment models:

```
from transformers import pipeline

sentiment_analyzer = pipeline(
    "sentiment-analysis",
    model="cardiffnlp/twitter-roberta-base-sentiment"
)

def analyze_sentiment(text):
    result = sentiment_analyzer(text[:512])

    return {
        'label': result[0]['label'],
        'score': result[0]['score']
    }

def batch_analyze_sentiment(texts):
    sentiments = []
    for text in texts:
        sentiment = analyze_sentiment(text)
        sentiments.append(sentiment)
    return sentiments

def sentiment_score_to_numeric(sentiment_label):
    mapping = {
        'POSITIVE': 0.8,
        'NEGATIVE': 0.2,
        'NEUTRAL': 0.5
    }
    return mapping.get(sentiment_label, 0.5)
```

7.6 Comprehensive NLP Pipeline

```
class ComprehensiveNLPPipeline:
    def __init__(self):
        self.nlp = spacy.load("en_core_web_sm")
        self.kb_model = KeyBERT(model='all-MiniLM-L6-v2')
        self.summarizer = pipeline(
            "summarization",
            model="facebook/bart-large-cnn"
        )
```

```
        self.sentiment_analyzer = pipeline(
            "sentiment-analysis",
            model="cardiffnlp/twitter-roberta-base-sentiment"
        )

    def process(self, text):
        doc = self.nlp(text)

        result = {
            'entities': extract_entities(doc),
            'keywords': extract_combined_keywords(text),
            'summary': generate_summary(text),
            'sentiment': analyze_sentiment(text),
            'topics': classify_topics(text),
            'tokens': [token.text for token in doc],
            'noun_chunks': [chunk.text for chunk in
               doc.noun_chunks]
        }

        return result
```

Chapter 8

Embedding Generation

8.1 Embedding Models

SYNAPSE supports multiple embedding models for vector representation of text.

8.1.1 OpenAI Text Embedding Ada-002

Production-grade embeddings from OpenAI:

```
import openai

class OpenAIEmbeddings:
    def __init__(self, api_key):
        openai.api_key = api_key
        self.model = "text-embedding-ada-002"

    def embed(self, text):
        response = openai.Embedding.create(
            input=text,
            model=self.model
        )
        return response['data'][0]['embedding']

    def embed_batch(self, texts, batch_size=100):
        embeddings = []
        for i in range(0, len(texts), batch_size):
            batch = texts[i:i+batch_size]
            response = openai.Embedding.create(
                input=batch,
                model=self.model
            )
            for item in response['data']:
                embeddings.append(item['embedding'])
        return embeddings
```

8.1.2 Sentence Transformers

Local, privacy-preserving embeddings:

```python
from sentence_transformers import SentenceTransformer

class LocalEmbeddings:
    def __init__(self, model_name='all-MiniLM-L6-v2'):
        self.model = SentenceTransformer(model_name)
        self.dimension = 384

    def embed(self, text):
        return self.model.encode(text, convert_to_numpy=True)

    def embed_batch(self, texts, batch_size=32):
        return self.model.encode(
            texts,
            batch_size=batch_size,
            convert_to_numpy=True,
            show_progress_bar=True
        )
```

8.2 Batch Processing

```python
class EmbeddingBatchProcessor:
    def __init__(self, model, batch_size=32, rate_limit=None):
        self.model = model
        self.batch_size = batch_size
        self.rate_limit = rate_limit

    def process_batch(self, items):
        texts = [item['text'] for item in items]
        embeddings = self.model.embed_batch(texts,
           self.batch_size)

        results = []
        for item, embedding in zip(items, embeddings):
            results.append({
                'item_id': item['id'],
                'embedding': embedding,
                'dimension': len(embedding)
            })

        return results

    def process_with_retry(self, items, max_retries=3):
        for attempt in range(max_retries):
            try:
                return self.process_batch(items)
            except Exception as e:
                if attempt == max_retries - 1:
                    raise
                wait_time = 2 ** attempt
                time.sleep(wait_time)
```

8.3 Rate Limiting

```
from time import time, sleep

class RateLimiter:
    def __init__(self, requests_per_minute=3000):
        self.rpm = requests_per_minute
        self.request_times = []

    def wait_if_needed(self):
        now = time()
        cutoff = now - 60

        self.request_times = [
            t for t in self.request_times if t > cutoff
        ]

        if len(self.request_times) >= self.rpm:
            sleep_time = 60 - (now - self.request_times[0])
            if sleep_time > 0:
                sleep(sleep_time)

        self.request_times.append(time())
```

8.4 pgvector Storage

Storing embeddings in PostgreSQL with pgvector extension:

```
from pgvector.django import VectorField
from django.db import models

class ItemEmbedding(models.Model):
    item = models.OneToOneField(
        'Item',
        on_delete=models.CASCADE,
        related_name='embedding'
    )
    vector = VectorField(dimensions=1536)
    model = models.CharField(max_length=100)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = 'item_embeddings'
        indexes = [
            models.Index(fields=['created_at']),
        ]

class ItemEmbeddingManager:
    @staticmethod
    def create_from_vector(item_id, vector, model_name):
```

```
        return ItemEmbedding.objects.create(
            item_id=item_id,
            vector=vector,
            model=model_name
        )

    @staticmethod
    def find_similar(query_vector, limit=10):
        from django.db.models import F
        from pgvector.django import CosineDistance

        return ItemEmbedding.objects.annotate(
            distance=CosineDistance(F('vector'), query_vector)
        ).order_by('distance')[:limit]
```

Chapter 9

Storage Strategy

9.1 PostgreSQL Schema

Multi-table schema for different data types:

```
class Item(models.Model):
    SOURCE_CHOICES = [
        ('hackernews', 'Hacker␣News'),
        ('github', 'GitHub'),
        ('arxiv', 'arXiv'),
        ('youtube', 'YouTube'),
        ('newsapi', 'NewsAPI'),
    ]

    source = models.CharField(
        max_length=20,
        choices=SOURCE_CHOICES
    )
    external_id = models.CharField(max_length=255)
    title = models.TextField()
    url = models.URLField(max_length=2000)
    content = models.TextField(blank=True)
    author = models.CharField(max_length=255, blank=True)
    published_at = models.DateTimeField(null=True)
    extracted_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = [['source', 'external_id']]
        indexes = [
            models.Index(fields=['source', 'published_at']),
            models.Index(fields=['extracted_at']),
        ]

class ItemMetadata(models.Model):
    item = models.OneToOneField(
        Item,
        on_delete=models.CASCADE,
        related_name='metadata'
    )
```

```
    entities = models.JSONField(default=list)
    keywords = models.JSONField(default=list)
    topics = models.JSONField(default=list)
    sentiment = models.JSONField(default=dict)
    summary = models.TextField(blank=True)
    quality_score = models.FloatField(default=0.5)
    language = models.CharField(max_length=10, default='en')
```

9.2 Redis Caching Strategy

Multi-tier TTL strategy for hot data:

```
import redis
import json
from datetime import timedelta

class RedisCache:
    def __init__(self, host='localhost', port=6379, db=0):
        self.redis_client = redis.Redis(
            host=host,
            port=port,
            db=db,
            decode_responses=True
        )

    def cache_trending(self, key, value, ttl_minutes=5):
        ttl = timedelta(minutes=ttl_minutes)
        self.redis_client.setex(
            f"trending:{key}",
            ttl,
            json.dumps(value)
        )

    def cache_article(self, key, value, ttl_minutes=30):
        ttl = timedelta(minutes=ttl_minutes)
        self.redis_client.setex(
            f"article:{key}",
            ttl,
            json.dumps(value)
        )

    def cache_search(self, key, value, ttl_minutes=10):
        ttl = timedelta(minutes=ttl_minutes)
        self.redis_client.setex(
            f"search:{key}",
            ttl,
            json.dumps(value)
        )

    def get_cached(self, key, key_type='article'):
        full_key = f"{key_type}:{key}"
```

```
        cached = self.redis_client.get(full_key)
        return json.loads(cached) if cached else None
```

9.3 S3 Media Storage

Storing large media files and documents:

```
import boto3
from datetime import datetime

class S3Storage:
    def __init__(self, bucket_name, region='us-east-1'):
        self.s3_client = boto3.client('s3', region_name=region)
        self.bucket = bucket_name

    def upload_document(self, file_path, item_id, source):
        key = f"documents/{source}/{item_id}/{file_path}"
        self.s3_client.upload_file(
            file_path,
            self.bucket,
            key
        )
        return f"s3://{self.bucket}/{key}"

    def upload_media(self, file_content, item_id, media_type):
        timestamp = datetime.utcnow().isoformat()
        key = f"media/{media_type}/{item_id}/{timestamp}"

        self.s3_client.put_object(
            Bucket=self.bucket,
            Key=key,
            Body=file_content
        )

        return f"s3://{self.bucket}/{key}"

    def generate_presigned_url(self, key, expiration=3600):
        return self.s3_client.generate_presigned_url(
            'get_object',
            Params={'Bucket': self.bucket, 'Key': key},
            ExpiresIn=expiration
        )
```

Chapter 10

Scheduler Design

10.1 Celery Beat Configuration

Celery Beat provides periodic task scheduling with timezone awareness:

```
from celery.schedules import crontab
from celery import Celery

app = Celery('synapse')

app.conf.beat_schedule = {
    'scrape-hackernews': {
        'task': 'tasks.scrape_source',
        'schedule': crontab(minute='*/30'),
        'args': ('hackernews',),
        'options': {'queue': 'scraping', 'priority': 7}
    },
    'scrape-github': {
        'task': 'tasks.scrape_source',
        'schedule': crontab(minute='0', hour='*/2'),
        'args': ('github',),
        'options': {'queue': 'scraping', 'priority': 5}
    },
    'scrape-arxiv': {
        'task': 'tasks.scrape_source',
        'schedule': crontab(minute='0', hour='*/6'),
        'args': ('arxiv',),
        'options': {'queue': 'scraping', 'priority': 3}
    },
    'scrape-youtube': {
        'task': 'tasks.scrape_source',
        'schedule': crontab(minute='0', hour='0'),
        'args': ('youtube',),
        'options': {'queue': 'scraping', 'priority': 2}
    },
    'trend-analysis': {
        'task': 'tasks.analyze_trends',
        'schedule': crontab(minute='0', hour='*/6'),
        'options': {'queue': 'analysis'}
```

```
    },
    'generate-weekly-report': {
        'task': 'tasks.generate_report',
        'schedule': crontab(day_of_week=0, hour=0, minute=0),
        'options': {'queue': 'reports'}
    },
}

app.conf.timezone = 'UTC'
```

10.2 Missed Task Handling

Handling tasks that were missed due to scheduler downtime:

```
from django.utils import timezone
from datetime import timedelta
import logging

logger = logging.getLogger(__name__)

class MissedTaskHandler:
    @staticmethod
    def detect_missed_tasks():
        now = timezone.now()
        missed = ScheduledTask.objects.filter(
            enabled=True,
            next_run__lt=now,
            last_run__lt=now - timedelta(hours=2)
        )
        return missed

    @staticmethod
    def execute_missed_tasks(max_retries=3):
        missed_tasks = MissedTaskHandler.detect_missed_tasks()

        for task in missed_tasks:
            retries = 0
            while retries < max_retries:
                try:
                    ScheduleManager.execute_task(task)
                    break
                except Exception as e:
                    retries += 1
                    if retries == max_retries:
                        logger.error(
                            f"Failed␣to␣execute:␣
                                {task.task_name}"
                        )
                    else:
                        time.sleep(2 ** retries)
```

Chapter 11

Data Quality

11.1 Validation Rules

Comprehensive validation rules for data integrity:

```
class DataQualityValidator:
    VALIDATION_RULES = {
        'title_length': {'min': 5, 'max': 500, 'weight': 0.2},
        'content_length': {'min': 50, 'max': 100000, 'weight':
           0.2},
        'url_format': {'pattern': r'^https?://', 'weight': 0.15},
        'published_date': {'max_age_days': 365, 'weight': 0.15},
        'author_present': {'required': False, 'weight': 0.1},
        'language': {'allowed': ['en'], 'weight': 0.1},
        'duplicate': {'allow_duplicates': False, 'weight': 0.1},
    }

    @classmethod
    def validate_item(cls, item):
        score = 0.0
        violations = []

        if len(item.get('title', '')) < 5:
            violations.append('Title␣too␣short')

        if len(item.get('content', '')) < 50:
            violations.append('Content␣too␣short')

        if not item.get('url'):
            violations.append('Missing␣URL')

        if violations:
            return {'score': 0.0, 'violations': violations}

        return {'score': 0.8, 'violations': []}
```

11.2 Quality Scoring

Multi-factor quality scoring system:

```
from datetime import datetime

class QualityScorer:
    def __init__(self):
        self.weights = {
            'completeness': 0.25,
            'accuracy': 0.25,
            'consistency': 0.20,
            'freshness': 0.15,
            'relevance': 0.15
        }

    def calculate_completeness(self, item):
        fields = ['title', 'url', 'content', 'author']
        filled = sum(1 for f in fields if item.get(f))
        return filled / len(fields)

    def calculate_freshness(self, published_at):
        days_old = (datetime.utcnow() - published_at).days
        if days_old < 7:
            return 1.0
        elif days_old < 30:
            return 0.8
        elif days_old < 365:
            return 0.5
        else:
            return 0.1

    def calculate_overall_score(self, item):
        completeness = self.calculate_completeness(item)
        freshness =
           self.calculate_freshness(item['published_at'])
        consistency = 0.7

        score = (
            completeness * self.weights['completeness'] +
            freshness * self.weights['freshness'] +
            consistency * self.weights['consistency']
        )

        return min(max(score, 0.0), 1.0)
```

11.3 Duplicate Detection Metrics

```
import logging
```

```python
logger = logging.getLogger(__name__)

class DuplicateMetrics:
    def __init__(self):
        self.total_items = 0
        self.duplicates_found = 0
        self.near_duplicates_found = 0

    def calculate_duplication_rate(self):
        if self.total_items == 0:
            return 0.0
        return (
            (self.duplicates_found +
             self.near_duplicates_found) /
            self.total_items
        )

    def log_metrics(self):
        rate = self.calculate_duplication_rate()
        logger.info(
            f"Total items: {self.total_items}, "
            f"Exact duplicates: {self.duplicates_found}, "
            f"Near duplicates: {self.near_duplicates_found}, "
            f"Duplication rate: {rate*100:.2f}%"
        )
```

Chapter 12

Open Source Libraries

12.1 Core Dependencies

SYNAPSE leverages production-grade open source libraries:

- **Scrapy 2.11**: Web scraping framework with middleware support
- **Celery 5.3**: Distributed task queue with Redis broker integration
- **spaCy 3.7**: Industrial-grade NLP with pre-trained models
- **transformers 4.36**: HuggingFace transformer models for NLP tasks
- **sentence-transformers 2.2**: Pre-trained sentence embedding models
- **KeyBERT 0.8**: BERT-based keyword extraction
- **langdetect 1.0**: Language detection library
- **datasketch**: MinHash and LSH for near-duplicate detection
- **pgvector 0.2**: PostgreSQL vector extension
- **redis-py 5.0**: Python Redis client
- **Flower 2.0**: Celery monitoring tool
- **BeautifulSoup4**: HTML/XML parsing
- **Playwright**: Browser automation for JS-heavy sites
- **boto3**: AWS S3 integration
- **Prometheus-client**: Metrics collection

12.2 Version Management

Dependencies are pinned to specific versions for reproducibility:

```
Scrapy==2.11.0
Celery==5.3.4
spacy==3.7.2
transformers==4.36.2
sentence-transformers==2.2.2
keybert==0.8.1
langdetect==1.0.9
datasketch==1.0.8
pgvector==0.2.4
redis==5.0.1
flower==2.0.1
beautifulsoup4==4.12.2
playwright==1.40.0
boto3==1.34.2
prometheus-client==0.19.0
```

Chapter 13

Pipeline Monitoring

13.1 Prometheus Metrics

Key performance indicators tracked via Prometheus:

```
from prometheus_client import Counter, Histogram, Gauge
import time

tasks_processed = Counter(
    'synapse_tasks_processed_total',
    'Total␣tasks␣processed',
    ['source', 'status']
)

scrape_success_rate = Gauge(
    'synapse_scrape_success_rate',
    'Web␣scraping␣success␣rate',
    ['source']
)

nlp_processing_time = Histogram(
    'synapse_nlp_processing_seconds',
    'NLP␣processing␣time␣in␣seconds',
    buckets=[0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
)

embedding_generation_rate = Counter(
    'synapse_embeddings_generated_total',
    'Total␣embeddings␣generated',
    ['model']
)

queue_depth = Gauge(
    'synapse_queue_depth',
    'Current␣task␣queue␣depth',
    ['queue_name']
)

duplicate_detection_count = Counter(
```

```
    'synapse_duplicates_detected_total',
    'Total duplicates detected',
    ['detection_method']
)
```

13.2 Alerting Rules

Prometheus alerting rules for production monitoring:

```
groups:
  - name: synapse_alerts
    rules:
      - alert: HighFailureRate
        expr: |
          (increase(synapse_tasks_processed_total{status="failed"}[5m])
             /
           increase(synapse_tasks_processed_total[5m])) > 0.1
        for: 5m
        annotations:
          summary: "High task failure rate detected"

      - alert: LowScrapeSuccessRate
        expr: synapse_scrape_success_rate < 0.8
        for: 10m
        annotations:
          summary: "Web scraping success rate below 80%"

      - alert: QueueDepthHigh
        expr: synapse_queue_depth > 10000
        for: 5m
        annotations:
          summary: "Task queue depth exceeding threshold"

      - alert: HighDuplicationRate
        expr: |
          (increase(synapse_duplicates_detected_total[1h]) /
           increase(synapse_tasks_processed_total[1h])) > 0.3
        for: 10m
        annotations:
          summary: "High duplicate detection rate"
```

13.3 Grafana Dashboards

Dashboard configuration for real-time monitoring:

```
DASHBOARD_CONFIG = {
    "dashboard": {
        "title": "SYNAPSE Pipeline Monitoring",
        "refresh": "30s",
        "panels": [
```

```
{
    "title": "Tasks␣Processed␣Rate",
    "targets": [{
        "expr":
           "rate(synapse_tasks_processed_total[5m])"
    }]
},
{
    "title": "Task␣Failure␣Rate",
    "targets": [{
        "expr":
           "rate(synapse_tasks_processed_total{status='failed'}[5
    }]
},
{
    "title": "NLP␣Processing␣Time",
    "targets": [{
        "expr": "synapse_nlp_processing_seconds"
    }]
},
{
    "title": "Queue␣Depth␣by␣Queue",
    "targets": [{
        "expr": "synapse_queue_depth"
    }]
},
{
    "title": "Duplicate␣Detection␣Rate",
    "targets": [{
        "expr":
           "rate(synapse_duplicates_detected_total[1h])"
    }]
},
{
    "title": "Data␣Quality␣Score␣Distribution",
    "targets": [{
        "expr": "histogram_quantile(0.95,␣
           quality_score)"
    }]
},
{
    "title": "Embeddings␣Generated",
    "targets": [{
        "expr":
           "increase(synapse_embeddings_generated_total[1h])"
    }]
},
{
    "title": "Scraper␣Success␣Rate␣by␣Source",
    "targets": [{
        "expr": "synapse_scrape_success_rate"
```

```
                }]
            }
        ]
    }
}
```

13.4 Health Checks

Endpoint for pipeline health monitoring:

```
import redis
from django.http import JsonResponse
from django.views.decorators.http import require_http_methods
import psycopg2

@require_http_methods(["GET"])
def health_check(request):
    health_status = {
        'status': 'healthy',
        'components': {},
        'timestamp': datetime.utcnow().isoformat()
    }

    try:
        redis_client = redis.Redis(host='localhost', port=6379)
        redis_client.ping()
        health_status['components']['redis'] = 'healthy'
    except Exception as e:
        health_status['components']['redis'] = 'unhealthy'
        health_status['status'] = 'degraded'

    try:
        from django.db import connection
        connection.ensure_connection()
        health_status['components']['database'] = 'healthy'
    except Exception as e:
        health_status['components']['database'] = 'unhealthy'
        health_status['status'] = 'unhealthy'

    try:
        from celery.app.control import Inspect
        inspect = Inspect()
        stats = inspect.stats()
        if stats:
            health_status['components']['celery'] = 'healthy'
        else:
            health_status['components']['celery'] = 'unhealthy'
            health_status['status'] = 'degraded'
    except Exception as e:
        health_status['components']['celery'] = 'unhealthy'
        health_status['status'] = 'degraded'
```

```
status_code = 200 if health_status['status'] == 'healthy'
    else 503
return JsonResponse(health_status, status_code=status_code)
```

Chapter 14

Conclusion

The SYNAPSE data pipeline represents a comprehensive, production-grade system for collecting, processing, and analyzing data from multiple web sources. By leveraging distributed task queues, advanced NLP models, and multi-tier storage strategies, SYNAPSE achieves both scalability and data quality.

Key architectural decisions enable:

1. **High Throughput**: Processing tens of thousands of items daily across multiple sources
2. **Data Quality**: Multi-stage validation, deduplication, and quality scoring
3. **Intelligence**: Advanced NLP transformations providing semantic understanding
4. **Search Capability**: Vector embeddings enable semantic similarity search
5. **Reliability**: Retry mechanisms, dead letter queues, and comprehensive monitoring
6. **Flexibility**: Modular architecture supports easy addition of new sources and processing stages

Continuous monitoring via Prometheus and Grafana ensures operational visibility, while comprehensive error handling and alerting maintain system reliability. The use of well-maintained open source libraries ensures long-term maintainability and community support.